



Tecnológico de Monterrey

Implementación de una técnica de aprendizaje máquina sin el uso de un framework.

Random Forest

Michelle Rergis Novelo A01798576

2 de Septiembre del 2026

Módulo 2. Inteligencia artificial avanzada para la ciencia de datos.

Profesor: Jorge Adolfo Ramírez Uresti

1. Introducción y Objetivo

Este reporte documenta la implementación de un algoritmo de Random Forest programado desde cero, sin usar bibliotecas de machine learning para el núcleo del modelo. El objetivo es clasificar la dieta de un dinosaurio en una de tres categorías — carnivorous, herbivorous u omnivorous — a partir de variables morfológicas (longitud corporal), temporales (periodo geológico en el que vivió) y de contexto taxonómico/geográfico (tipo anatómico, gran división taxonómica, región del hallazgo).

1.1 ¿Qué es un Random Forest?

Un Random Forest (bosque aleatorio) es un algoritmo de aprendizaje supervisado de tipo ensamble (ensemble learning): en vez de entrenar un único modelo, entrena muchos árboles de decisión independientes entre sí y combina sus predicciones para dar una respuesta final. Fue formalizado por Leo Breiman en 2001, y pertenece a la familia de métodos que buscan reducir la varianza de un modelo base inestable (el árbol de decisión) sin sacrificar demasiado su capacidad predictiva.

Un árbol de decisión individual es fácil de interpretar y puede ajustarse muy bien a los datos de entrenamiento, pero es un modelo de alta varianza: pequeños cambios en los datos (o incluso entrenarlo dos veces con distinto orden de muestras) pueden producir árboles con estructuras y predicciones muy distintas, lo que se traduce en sobreajuste (overfitting) y mala generalización a datos nuevos. El Random Forest ataca este problema con dos fuentes de aleatoriedad que se combinan:

- Bagging (bootstrap aggregating): cada árbol se entrena con una muestra bootstrap distinta -es decir, un remuestreo con reemplazo del mismo tamaño que el conjunto de entrenamiento original-, de modo que cada árbol ve una versión ligeramente distinta de los datos.
- Selección aleatoria de variables: en cada división (nodo) de cada árbol, sólo se evalúa un subconjunto aleatorio de las variables disponibles (no todas), en vez de buscar siempre la mejor división posible entre todas las variables.

Esta doble aleatoriedad hace que los árboles no estén correlacionados entre sí: cada uno comete errores distintos, ligados al ruido particular de su muestra y de las variables que le tocaron evaluar. Al final, el bosque combina las predicciones de todos los árboles -por voto de mayoría en clasificación, o por promedio en regresión-, y como los errores individuales no están correlacionados, tienden a cancelarse: el resultado es un modelo mucho más estable y con mejor capacidad de generalización que cualquier árbol individual, manteniendo al mismo tiempo la capacidad de los árboles de capturar relaciones no lineales entre variables numéricas y categóricas sin necesidad de escalarlas o normalizarlas.

Además de la predicción en sí, el proceso de construcción del bosque produce subproductos útiles para el análisis: la importancia de cada variable (cuanto contribuye, en promedio, a reducir la impureza de las clases a lo largo de todos los árboles) y una estimación interna del error de generalización a partir de las muestras “out-of-bag” (los datos que, por el muestreo con reemplazo, quedaron fuera de la muestra bootstrap de un árbol dado), sin necesidad de un conjunto de validación separado.

1.2 Alcance de esta implementación

El núcleo algoritmo está implementado únicamente con la biblioteca estándar de Python (math, random, collections). Las herramientas externas (pandas, numpy, matplotlib, seaborn) se usan solo para la carga y limpieza de datos, el análisis exploratorio y la generación de gráficas; la única excepción es scikit-learn, utilizada exclusivamente para calcular y graficar la matriz de confusión con el formato estándar de la librería (sklearn.metrics.confusion_matrix y ConfusionMatrixDisplay), mientras que precision, recall y F1 se siguen calculando a mano a partir de esa matriz.

2. Dataset: entrenamiento y prueba

2.1 Fuente y variables

El dataset utilizado es dinosaurs.csv, derivado del “*Dino Directory*” del Natural History Museum (disponible en Kaggle como “Jurassic Park - The Exhaustive Dinosaur Dataset”). El archivo original trae texto libre y no está listo para el modelo (por ejemplo, length viene como "8.0m" y period cómo "Early Jurassic 199-189 million years ago"), por lo que el script deriva las variables de entrada antes de entrenar:

Variable	Tipo	Origen (columna cruda)
length_m	Numérica	'length' (texto, ej. "8.0m") parseado con regex
max_ma / min_ma	Numérica	'period' (texto, ej. "Early Jurassic 199-189 mya")
type	Categórica	'type' (sauropod, theropod, ceratopsian, etc.)
class	Categórica	'taxonomy': segundo token (Saurischia / Ornithischia)
lived_in	Categórica	'lived_in' (país / región del hallazgo)
diet (target)	Categórica (3 clases)	'diet': carnivorous / herbivorous / omnivorous

Tabla 1. Variables derivadas y usadas por el modelo.

Antes del split, se descartaron los registros cuya dieta no sea una de las tres clases válidas (se eliminaron valores como “unknown” o combinaciones ambiguas) y las filas donde no fue posible derivar alguna variable necesaria (longitud, periodo o taxonomía malformados o ausentes).

2.2 Partición entrenamiento / prueba

La partición se realizó con una función de split estratificado implementada a mano (`train_test_split_stratified`), que **reserva el 20% de cada clase para prueba y el 80% restante para el entrenamiento**, preservando en ambos conjuntos la misma proporción de clases que existe en el dataset completo. Esto es importante porque se identificó al inicio (a partir de la distribución real), las clases están desbalanceadas (hay muchos más registros herbívoros y carnívoros que omnívoros); sin estratificación, un split aleatorio simple podría dejar muy pocos (o ningún) ejemplo de la clase minoritaria en el conjunto de prueba.

El conjunto de prueba obtenido (el que se reporta en la sección 4): 57 dinosaurios en total — 18 carnívoros, 34 herbívoros y 5 omnívoros (según los totales por fila de la matriz de confusión obtenida). El resto del dataset limpio se usa como conjunto de entrenamiento del bosque.

2.5. Flujo general del algoritmo

El siguiente diagrama resume, paso a paso, el flujo completo del script desde que se ejecuta `main()` hasta que termina: la preparación de datos, el entrenamiento del algoritmo (con el ciclo interno de construcción de cada árbol) y la evaluación/generación de resultados.

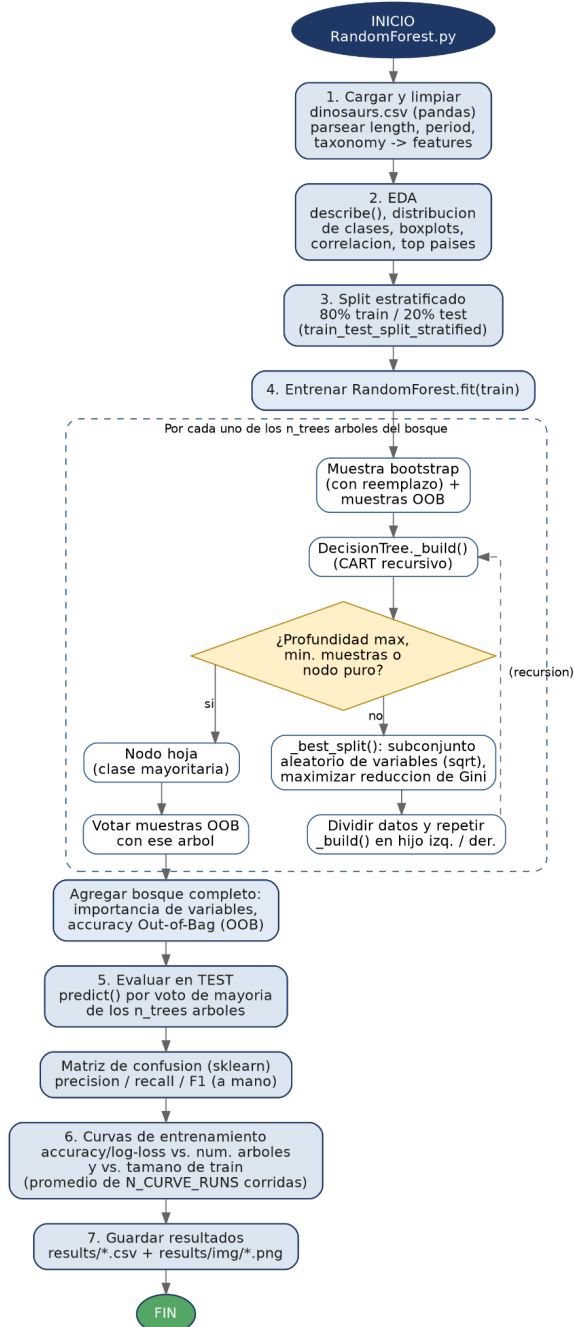


Figura 0. Diagrama de flujo del pipeline implementado en RandomForest.py

El programa arranca en `main()` y sigue un flujo lineal de siete etapas:

Primero (1) `load_dataframe()` lee `dinosaurs.csv` con `pandas` y deriva las variables que usará el modelo -`length_m`, `max_ma`, `min_ma`, `class`, `type` y `lived_in` - a partir de las columnas de texto libre originales, descartando filas con dieta inválida o con algún dato faltante.

Después (2) *run_eda()* genera las tablas y gráficas descriptivas (distribución de clases, boxplots, correlación, países) que se muestran en la sección 5.2, sin afectar en nada al modelo -es puramente exploratorio-.

Con los datos ya limpios, (3) *train_test_split_stratified()* separa 80% para entrenamiento y 20% para prueba, manteniendo la proporción de las tres dietas en ambos conjuntos.

La etapa (4) es el corazón del algoritmo: *RandomForest.fit(train)* recorre un ciclo de *n_trees* iteraciones (60 en la ejecución reportada) y, en cada una, repite el mismo procedimiento. Primero toma una muestra bootstrap del conjunto de entrenamiento (con reemplazo, del mismo tamaño que *train*) y guarda aparte los ejemplos que no cayeron en esa muestra (out-of-bag). Con esa muestra bootstrap, construye un *DecisionTree* llamando de forma recursiva a *_build()*: en cada nodo revisa si se cumple algún criterio de paro (profundidad máxima, muy pocas muestras para seguir dividiendo, o un nodo ya puro); si no se cumple, *_best_split()* elige un subconjunto aleatorio de variables (tamaño $\sqrt{\text{total}}$), evalúa todas las divisiones posibles con esas variables y se queda con la que más reduce la impureza de Gini, y el árbol se construye recursivamente sobre los dos subconjuntos (izquierdo/derecho) resultantes. Cuando se cumple el criterio de paro, el nodo se vuelve hoja con la clase mayoritaria de ese subconjunto. Una vez terminado cada árbol, se acumula su contribución a la importancia de variables del bosque y se registra su voto sobre las muestras out-of-bag que le tocaron, lo que al final del ciclo permite calcular el accuracy Out-of-Bag sin usar el conjunto de prueba.

Terminado el entrenamiento, (5) el random forest se evalúa sobre el conjunto de prueba: *predict()* hace que cada uno de los *n_trees* árboles emita su propia predicción para cada dinosaurio de prueba, y la predicción final del bosque es la clase con más votos (mayoría simple). Con esas predicciones se calcula la matriz de confusión (con *scikit-learn*) y, a partir de ella, precisión, recall y F1 por clase (a mano).

En (6), para diagnosticar overfitting, el script repite todo el proceso de entrenamiento/evaluación varias veces -variando el número de árboles y, por separado, el tamaño del conjunto de entrenamiento- y promedia *N_CURVE_RUNS* corridas con distinta semilla por cada punto, para graficar cómo evolucionan el accuracy y la log-loss de entrenamiento vs.

prueba. Finalmente, (7) todos los resultados (métricas, matriz de confusión, importancia de variables y las gráficas) se guardan en la carpeta results/, qué es lo que se resume en la sección 5 de este reporte.

3. Conceptos y justificación del algoritmo

Los siguientes conceptos sustentan las decisiones de diseño del algoritmo:

3.1 Árbol de decisión e impureza de Gini

Cada árbol del bosque es un CART (Classification And Regression Trees) binario que, en cada nodo, evalúa posibles divisiones (un punto de corte para variables numéricas, o “es igual a esta categoría / no lo es” para variables categóricas) y elige la que maximiza la reducción de impureza de Gini, definida como:

$$I_G(p) = 1 - \sum_{i=1}^J p_i^2 \quad (1.)$$

—donde J es el número total de clases. p_i Es la probabilidad (o proporción) de elementos que pertenecen a la clase i en ese nodo— sobre las proporciones p_c de cada clase en el nodo. Un nodo puro (impureza 0) o que no cumple los mínimos de muestras se convierte en hoja, cuya predicción es la clase mayoritaria.

3.2 Bagging (bootstrap aggregating) y muestreo aleatorio de variables

El Random Forest reduce la varianza de un árbol individual combinando muchos árboles entrenados sobre muestras bootstrap distintas (muestreo con reemplazo del mismo tamaño que el set de entrenamiento) y, además, restringiendo en cada división a un subconjunto aleatorio de variables (aquí, `max_features="sqrt"`, es decir raíz cuadrada del número total de variables, ~2 de 6). Esta doble aleatoriedad decorrelaciona los árboles entre sí: si un árbol se equivoca por sobre ajustarse al ruido de su muestra particular, es poco probable que la mayoría de los otros árboles

cometan el mismo error, por lo que la votación por mayoría del bosque generaliza mejor que cualquier árbol individual.

3.3 Balance sesgo-varianza (bias-variance tradeoff)

Los hiperparámetros del random forest (`max_depth=10`, `min_samples_split=10`, `min_samples_leaf=4`) controlan directamente este balance: árboles muy profundos y sin restricciones tienden a memorizar el ruido del conjunto de entrenamiento (**varianza alta, sesgo bajo => overfitting**), mientras que árboles demasiado restringidos no logran capturar los patrones reales (**sesgo alto => underfitting**). Limitar la profundidad y exigir un mínimo de muestras por hoja regulariza cada árbol individual; el bagging regulariza adicionalmente al promediar el voto de varios árboles regularizados.

3.4 Evaluación Out-of-Bag (OOB) como validación interna

Cada muestra bootstrap deja, en promedio, alrededor de un 37% de los ejemplos de entrenamiento fuera de la muestra (out-of-bag) para ese árbol específico. El script aprovecha esos ejemplos para estimar el desempeño del bosque usando sólo datos de entrenamiento nunca vistos por cada árbol individual, sin tocar el conjunto de prueba — una forma de validación interna conceptualmente cercana a la validación cruzada, pero que aprovecha la propia estructura del bagging en vez de dividir el set de entrenamiento en folds.

3.5 Importancia de variables

Cada vez que una variable se usa para dividir un nodo, se acumula la reducción de impureza que produjo esa división, ponderada por el número de muestras en el nodo. Al normalizar esa suma entre todos los árboles se obtiene la importancia relativa de cada variable, que ayuda a interpretar cuáles características morfológicas/taxonómicas/geográficas pesan más en la predicción de la dieta.

3.6 Curvas de entrenamiento y diagnóstico de overfitting

Para diagnosticar si el modelo está sobre ajustando o si le faltan datos/árboles, el script genera curvas de accuracy y log-loss en entrenamiento vs. prueba (equivalente al análisis de curvas de aprendizaje accuracy/loss por época de otros modelos, usando aquí el número de árboles y el tamaño del set de entrenamiento como eje de progreso). Como el bagging tiene varianza notable con pocos árboles o pocos datos, cada punto de la curva es el promedio de varias corridas con distinta semilla (incluyendo un split train/test distinto en cada corrida), para obtener una curva representativa y no una única corrida ruidosa.

4. Resultados obtenidos

4.1 Configuración del bosque entrenado

Hiper Parámetro	Valor
Número de árboles (n_trees)	60
Profundidad máxima (max_depth)	10
Min. muestras para dividir (min_samples_split)	10
Min. muestras por hoja (min_samples_leaf)	4
Variables evaluadas por division (max_features)	$\sqrt{6} = 2$, elegidas al azar en cada nodo
Criterio de división	Impureza de Gini (reducción ponderada)
Muestreo de datos por árbol	Bootstrap (bagging) con reemplazo, tamaño = n del set de entrenamiento
Semilla aleatoria (RANDOM_SEED)	42
Partición entrenamiento / prueba	80% / 20%, estratificada por clase

Tabla 2. Hiperparametros usados en la ejecución.

4.2 Análisis exploratorio relevante

La correlación entre las variables numéricas derivadas muestra, una relación casi perfecta entre *max_ma* y *min_ma* (0.99, ambas provienen del mismo rango de periodo geológico) y una correlación prácticamente nula entre la longitud corporal y las variables temporales — es decir, el tamaño de un dinosaurio no depende de en qué época vivió, por lo que ambas familias de variables aportan información independiente al modelo.

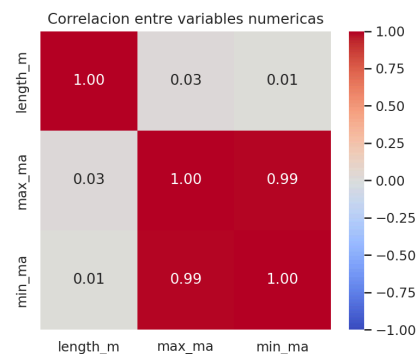


Figura 1. Correlacion entre las variables numericas (*length_m*, *max_ma*, *min_ma*).

Por otro lado, la distribución geográfica de los hallazgos (variable categórica *lived_in*) está dominada por Estados Unidos, China y Mongolia, seguidos de Argentina y Reino Unido — reflejo de donde históricamente se ha concentrado la explotación paleontológica, y una variable categórica con muchas categorías poco frecuentes, lo que el árbol maneja evaluando divisiones binarias “es esta categoría / no lo es” en cada nodo.

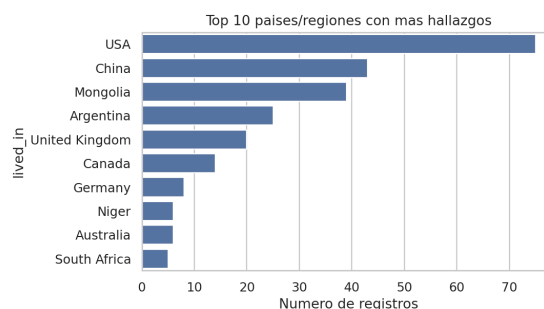


Figura 2. Top 10 países/regiones con más hallazgos de dinosaurios en el dataset.

4.3 Matriz de confusión (conjunto de prueba)

La matriz de confusión se calculó con `sklearn.metrics.confusion_matrix` / `ConfusionMatrixDisplay` sobre las 57 predicciones del conjunto de prueba (datos nunca usados para entrenar ningún árbol del bosque):

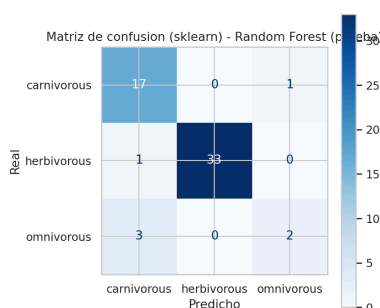


Figura 3. Matriz de confusión del Random Forest sobre el conjunto de prueba.

Real \ Predicho	carnivorous	herbivorous	omnivorous
carnivorous	17	0	1
herbivorous	1	33	0
omnivorous	3	0	2

Tabla 3. Matriz de confusión en formato tabular (filas = clase real, columnas = clase predicha).

La diagonal principal (17, 33 y 2 predicciones correctas para carnivorous, herbivorous y omnivorous respectivamente) concentra la gran mayoría de los casos. Los errores están casi todos ligados a la clase omnivorous: 3 de los 5 dinosaurios realmente omnívoros fueron clasificados como carnivorous, y solo 1 caso se confundió entre carnivorous y herbivorous en cada dirección. Ningún herbívoro real fue clasificado como omnívoro ni viceversa en más de un caso, lo que sugiere que el random forest distingue con claridad los patrones morfológicos de un herbívoro

típico, pero le cuesta separar la dieta omnívora — la más escasa y probablemente la más heterogénea morfológicamente — de la carnívora.

4.4 Métricas de clasificación

A partir de la matriz de confusión se calcularon a mano (sin sklearn) precision, recall y F1-score por clase, además del accuracy global y los promedios macro y ponderado (weighted):

Clase	Precisión	Recall	F1-score	Support
carnivorous	0.8095	0.9444	0.8718	18
herbivorous	1.0000	0.9706	0.9851	34
omnivorous	0.6667	0.4000	0.5000	5
Accuracy			0.9123	57
Macro avg	0.8254	0.7717	0.7856	57
Weighted avg	0.9106	0.9123	0.9066	57

Tabla 4. Precision, recall, F1-score y support por clase, más accuracy y promedios macro/weighted.

El accuracy global en el conjunto de prueba fue de 0.9123 (52 de 57 predicciones correctas). Se reportan también precision y recall porque el accuracy por sí solo puede ser engañoso en un problema con clases desbalanceadas como este (omnivorous tiene solo 5 ejemplos de prueba frente a 34 de herbivorous): un modelo que nunca prediga omnivorous podría tener accuracy alto igual, pero recall de 0 en esa clase.

El recall de 0.40 en omnivorous confirma que, aunque el accuracy global es alto, el modelo recupera menos de la mitad de los casos omnívoros reales; su precisión de 0.6667 indica además que, de las veces que el modelo predice omnivorous, 1 de cada 3 es en realidad un error. El F1-score (media armónica de precision y recall) resume ambos efectos por clase y es notablemente más bajo para omnivorous (0.50) que para carnivorous (0.87) o herbivorous (0.99, la clase más representada y mejor separada).

4.5 Curvas de entrenamiento (accuracy y loss vs. numero de arboles)

Para diagnosticar overfitting/underfitting se entrenaron bosques con distinto número de árboles (1 a 60), promediando 5 corridas con semillas distintas por punto (cada corrida con su propio split train/test):

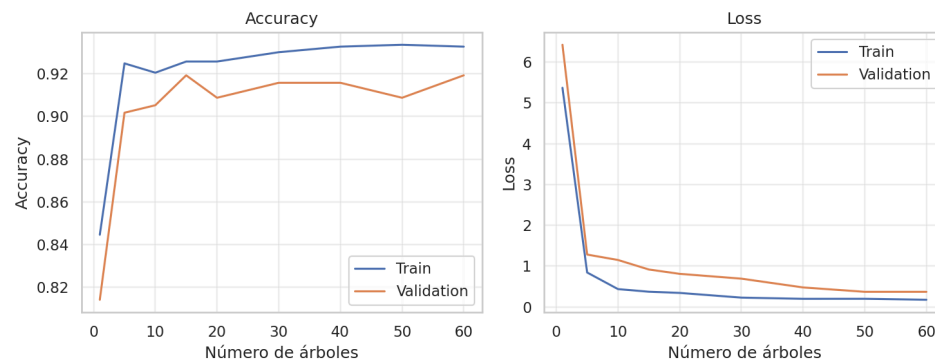


Figura 4. Accuracy y log-loss en entrenamiento vs. prueba, en función del número de árboles del bosque.

Con muy pocos árboles (1) el accuracy de entrenamiento y de prueba son similares y bajos (~0.84 y ~0.81 respectivamente) y la *log-loss* es alta en ambos, típico de un modelo poco entrenado (alto sesgo, bagging insuficiente). A partir de 5 árboles el accuracy de entrenamiento sube y se estabiliza alrededor de 0.92-0.93, mientras que el de prueba (validation) oscila entre 0.90 y 0.92 sin desviarse mucho del de entrenamiento. La brecha entre ambas curvas es pequeña y estable a partir de ~10-15 árboles, y la *log-loss* de prueba se mantiene relativamente plana (no vuelve a subir) mientras la de entrenamiento sigue bajando levemente — un indicio de sobreajuste leve/controlado, consistente con el uso de `max_depth=10` y `min_samples_leaf=4` como regularización, y no de un overfitting severo.

5. Análisis y conclusión del desempeño

El Random Forest implementado desde cero alcanza un desempeño sólido en el conjunto de prueba: 91.2% de accuracy global, con un F1 ponderado de 0.9066 y un F1 macro de 0.7856. La diferencia entre ambos promedios (macro vs. weighted) es la evidencia más clara del principal reto del modelo: el desbalance de clases. Las clases mayoritarias (herbivorous y carnivorous, con

34 y 18 ejemplos de prueba) se clasifican muy bien -F1 de 0.99 y 0.87 respectivamente-, mientras que la clase minoritaria (omnivorous, solo 5 ejemplos) tiene un F1 de apenas 0.50, arrastrando el promedio macro hacia abajo aun cuando el accuracy global se ve alto.

Las curvas de entrenamiento respaldan que el modelo está razonablemente bien regularizado: la brecha train/validation es moderada y no crece de forma descontrolada conforme aumenta el número de árboles, por lo que el desempeño limitado en la clase omnivorous no parece deberse principalmente a overfitting del random forest, sino a que hay muy pocos ejemplos de esa clase (tanto en entrenamiento como en prueba) para que el modelo aprenda un patrón morfológico/taxonómico distintivo, y a que -biológicamente- un dinosaurio omnívoro puede compartir rasgos con carnívoros y herbívoros por igual, lo que dificulta trazar una frontera de decisión clara con las variables disponibles.

Como conclusión, la implementación manual (árbol CART con Gini, bagging, selección aleatoria de variables, voto por mayoría e importancia de variables) logra un desempeño competitivo y coherente con el comportamiento esperado de un Random Forest: mejora sustancialmente respecto a un solo árbol o a pocos árboles, se estabiliza tras un número moderado de estimadores, y sus errores se concentran de forma explicable en la clase con menos datos disponibles. Como trabajo futuro, el desempeño en la clase omnivorous podría mejorar recolectando o generando más ejemplos de esa clase (o aplicando técnicas de balanceo como sobremuestreo), y probando variables adicionales (por ejemplo, features derivadas de la taxonomía completa más allá del segundo token) que ayuden a separarla mejor de carnívoros.

6. Referencias

Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140. <https://doi.org/10.1007/BF00058655>

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>

kjanjua. (s.f.). Jurassic Park - The Exhaustive Dinosaur Dataset [Conjunto de datos]. Kaggle. <https://www.kaggle.com/datasets/kjanjua/jurassic-park-the-exhaustive-dinosaur-dataset>

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>